Doggy App — SOC 2 Security Readiness Self-Assessment

Cloud security posture assessment of an AWS-hosted, cloud-native application

---

Executive Summary

This document presents a SOC 2 Security (Common Criteria) readiness self-assessment of the Doggy App environment — a cloud-native application running on AWS (EKS, RDS, CloudFormation-defined networking, and a GitLab-driven CI/CD pipeline). The assessment used Prowler for a point-in-time automated evaluation of the AWS environment, supplemented by manual review of the infrastructure-as-code and architecture. This is a self-assessment intended to gauge readiness against the Trust Services Criteria; it is not an independent audit or an attestation.

The environment is a single-developer project still in active development, which materially shapes the risk analysis throughout: several criteria concerned with organizational governance and segregation of duties are addressed narratively rather than as technical controls, and a number of continuous-monitoring services are consciously deferred on cost grounds with documented compensating controls. High-severity findings — chiefly network boundary exposures — were prioritized for remediation. Lower-severity and cost-driven items are captured in the remediation roadmap at the end of this report rather than resolved, reflecting a risk-based sequencing appropriate to a non-production environment.

Organizational Tier Assessment

The organization is assessed as Tier 1 (Partial). Organizational objectives are not formally specified and prioritization of risk remediation is ad hoc. There is awareness of cybersecurity risks informing architectural design; however, the process of incorporating best practices is irregular and handled on a case-by-case basis. This tier rating is consistent with — and appropriate for — a single-developer project in active development, and is not itself treated as a deficiency to be remediated.

Scope & Methodology

Scope covers the AWS account hosting the Doggy App: IAM and identity configuration, VPC networking (subnets, security groups, NACLs), the EKS cluster and its supporting resources, storage (S3, RDS), and logging/monitoring services. The assessment was performed with Prowler against its SOC 2 compliance mapping, cross-referenced with the AWS Foundational Security Best Practices and CIS AWS Foundations Benchmark checks that underlie it, and supplemented by manual inspection of the CloudFormation templates and running resources.

Governance Criteria (CC1–CC5)

Note on methodology for CC1–CC5. The COSO-derived governance criteria in CC1–CC5 concern organizational structures, communication lines, fraud risk, and monitoring responsibilities — controls that automated scanning cannot meaningfully evaluate. Prowler maps certain technical findings to these criteria, but those findings (IAM privilege, CloudTrail, GuardDuty, node

permissions) are properly assessed as logical-access and system-operations controls and are addressed under CC6 and CC7 below. For this single-developer project, the governance criteria are addressed narratively.

CC1.3 / CC2.1 / CC4.2 *Control environment, communication, and monitoring responsibilities.*

In a single-developer environment the chain of authority, communication, and corrective-action responsibility begins and ends with one person, so the reporting-line and board-oversight controls these criteria envision are not applicable in their standard form. The substantive equivalent — ensuring control deficiencies are surfaced and acted upon — is met not through organizational process but through automated detection, alerting, and logging in the AWS environment. The maturity of that tooling is assessed under CC7.

CC3.1–CC3.4 *Risk identification and assessment; assessing change that impacts internal control.*

Risk identification for this environment is performed through this assessment itself: Prowler surfaces misconfigurations, which are then triaged, rated, and either remediated or accepted. Because the platform has a single operator, configuration changes are made and known by that operator; the residual gap is the absence of automated change-tracking (AWS Config), which is a cost-driven accepted risk documented under CC7.1. The specific technical findings Prowler associated with these criteria are assessed in their proper control families below.

CC5.2 *Selection and development of general control activities over technology.*

Prowler flagged the absence of the CIS-recommended set of monitoring alarms for security-relevant API activity — changes to S3 buckets, security groups, IAM policies, VPCs, network gateways, NACLs, and CloudTrail itself. These are CloudWatch metric filters and alarms over CloudTrail as a log source; they provide tamper-detection for changes originating inside or outside the account. Implementing them depends on CloudTrail being enabled (see CC7.2) and is best sequenced alongside detective controls. Priority: medium.

CC6 — Logical & Physical Access

CC6.1 *The entity implements logical access security software, infrastructure, and architectures over protected information assets.*

This criterion returned a passing score; the following describes the protections in place. The application database resides in a private subnet behind a security group that accepts traffic only from the application backend. The application uses parameterized frameworks to prevent injection attacks. Stored credentials are hashed using an industry-standard algorithm, and the database contents are encrypted at rest by default under AWS-managed encryption. Traffic between the load balancer and clients is encrypted with TLS, mitigating man-in-the-middle attacks. Intra-cluster traffic could additionally be encrypted; this is assessed as low priority for the current environment.

CC6.2 *Registration and authorization of new users prior to granting access.*

The application does not administer end-user accounts, so this criterion is not applicable to the current system boundary.

CC6.3 *The entity authorizes, modifies, or removes access based on roles and responsibilities, giving consideration to least privilege and segregation of duties.*

Roles delegated to components within the project are designed around least privilege: each part of the architecture was granted the minimal permissions required for its function. Two

exceptions were identified. First, an AWS-managed policy and an Identity Center administrative user each grant '*:*' privileges; the administrative user authenticates via 2FA and manages the EKS cluster from a management bastion, but does not require full administrative rights. The remediation is to replace broad grants with scoped policies providing only the permissions needed to manage each project's infrastructure, and to organize policies by project group. Second, the EKS node role carries overly permissive rights that enable privilege-escalation paths; these were introduced during a debugging cycle and can be removed from the inline policy without functional impact. Priority: high for the node-role reduction (low effort), medium for policy grouping.

CC6.6 *The entity implements logical access security measures to protect against threats from sources outside its system boundaries.*

Assessment of the network boundary revealed an over-reliance on security groups as the sole enforcement layer, with three compounding weaknesses. Together they tell a single story: the boundary depended on one control type, a platform-managed component punched a hole in it, and the layer behind it was permissive by default, so nothing caught the gap.

Public subnet IP assignment. Public subnets were configured to auto-assign public IP addresses, meaning any resource launched into them would silently become internet-reachable. No resources currently reside in these subnets, but a misbehaving component could provision one there. All intended internet-facing components (the Doggy App and the GitLab instance) are fronted by an Application Load Balancer, so external reachability is a deliberate choice mediated by an entity that can carry its own security group. The subnets have been reconfigured to disable automatic public-IP assignment (MapPublicIpOnLaunch = false).

NACL configuration. Network ACLs were left at their permissive default, allowing ingress on administrative ports including SSH and the cluster ports. Because NACLs are stateless, explicit deny entries were written into the IaC for administrative ports specifically, rather than broadly restricting the NACL — which would risk breaking ephemeral-port return traffic and the NAT path. Primary enforcement remains at the security-group layer per AWS design guidance, with the NACL denies providing subnet-level defense in depth for admin ports.

Cluster security group exposure (primary finding). A security group provisioned by the EKS cluster itself permitted ingress from any IP (0.0.0.0/0) to all ports, leaving node interfaces open to internet-wide port scanning. Critically, this was a platform-managed security group, not one defined in the CloudFormation templates — demonstrating a mismatch between the intended state described by the IaC and the resources actually provisioned in production. Because security groups are evaluated as a permissive union, the additional restrictive security group attached to the cluster provided no compensating control; the exposure had to be removed at its source. Remediation used the AWS Load Balancer Controller (installed via the AWS-provided Helm chart), which manages security-group rules to match services as the orchestrator provisions and de-provisions them, allowing finely scoped ingress in place of the blanket rule. Priority: high.

CC6.7 *The entity restricts and protects information in transmission, movement, and removal.*

This criterion returned a passing score. Client-facing traffic is protected by TLS, and the certificate is configured is provisioned from AWS and set to autorenew.

CC6.8 *The entity implements controls to prevent or detect and act upon the introduction of unauthorized or malicious software.*

Detective coverage for unauthorized activity depends on enabling GuardDuty, which is assessed

and recommended under CC7.2.

CC7 — System Operations

CC7.1 *Detection and monitoring to identify configuration changes introducing vulnerabilities and susceptibility to newly discovered vulnerabilities.*

The application stack collects logs and metrics via Prometheus and Loki, and simple alerts have been configured so that application-logic errors trigger a notification. These have provided limited but real value in identifying unsophisticated attacks such as directory brute-forcing against the application. A deliberate expansion of this alerting would be required for deeper attack visibility; given the investment involved and the environment's development status, this is assessed as low priority.

Configuration-change monitoring (AWS Config). The AWS Config recorder is disabled. Continuous configuration tracking would improve auditability and could surface change-based indicators of attack; however, this is an accepted risk. Config bills per configuration item, and an environment with an active CI/CD pipeline and EKS generates a high volume of configuration items (each pipeline run and each container scale event produces changes), so the cumulative cost — especially as further projects are added — would exceed the benefit for a non-production environment. Compensating controls: recurring point-in-time assessment via Prowler, and CloudFormation drift detection, which surfaces divergence from intended state at no additional cost. Would be enabled in production, scoped to security-relevant resource types.

CC7.2 *The entity monitors system components for anomalies indicative of malicious acts, natural disasters, and errors; anomalies are analyzed to determine whether they represent security events.*

Threat detection (GuardDuty). GuardDuty is not enabled. As a detective control, it analyzes CloudTrail events, VPC Flow Logs, and DNS logs to identify anomalous behavior; without it, a compromise could occur with no indication other than the resulting bill. At single-developer scale the foundational-tier cost is minimal (single-digit dollars per month) and the value is high. Recommendation: enable the foundational tier, scoped to one region with protection plans off to avoid the vCPU-based Runtime Monitoring cost. Priority: high.

Foundational logging (CloudTrail). CloudTrail is not enabled in any region, meaning API calls occur without an auditable trail, alerting, or forensic record. A single multi-region trail delivering management events to S3 is effectively free, and the storage cost for the resulting logs is trivial. This is foundational to nearly every other detective control in this report. Priority: high.

Network flow visibility (VPC Flow Logs). VPC Flow Logs are absent. Without them there is no network-traffic record to correlate against GuardDuty findings — flow logs are what make anomaly alerts actionable. The remediation is to build flow logging into the CloudFormation VPC definition, delivering to a bucket with short retention or a Glacier transition to bound storage cost. Collecting only rejected traffic was considered as a cheaper option but rejected: events of interest typically fire on accepted requests, so rejected-only logging would not meet the need. This is a balance between meaningful visibility and cost; the plan is to mitigate and then accept the residual risk. Priority: medium.

Service and DNS logging. S3 buckets, Elastic Load Balancers, and the Route 53 hosted zone warrant logging to support post-incident analysis. GuardDuty flags suspicious activity but does not retain a log trail for investigation; feeding these logs to S3 provides that record at low cost given the environment's light traffic. This is particularly relevant for the Route 53 hosted zone, since DNS queries probing the network may indicate an attack yet fall outside GuardDuty's

default coverage. Priority: medium.

Targeted CloudWatch alerting. Three specific detections are recommended via CloudWatch: (1) monitoring for calls to Amazon Bedrock — the system does not use Bedrock, so any invocation would indicate LLMjacking; (2) detection of brute-force attempts against the AWS Management Console; and (3) flagging of potentially malicious DNS queries.

The addition of detection (guard duty), an audit trail (CloudTrail) and strategic alerts (CloudWatch) would buffer the the environment controls and delegate monitoring responsibility into automated processes as mentioned in CC1.3/CC2.1 and CC4.2. The need for detection controls as another fall back is apparent as seen in section CC6.6. Priority: medium.

CC7.3 *The entity evaluates security events to determine whether they represent security incidents.*

No notable security events have occurred. At present, event evaluation is limited to monitoring the AWS budget as a coarse anomaly signal; the detective controls recommended under CC7.2 are prerequisites for meaningful event evaluation.

CC7.4 *The entity responds to identified security incidents via a defined incident response program.*

The current incident-response posture leverages the environment's disposability: the entire system is defined in IaC and can be torn down and rebuilt, so for a non-critical application in development, destroying and redeploying the stack is an acceptable response to compromise. Database backups will be required once the architecture is migrated onto RDS and data integral to the application's function is stored; the current data store (EFS) was chosen to minimize cost during development, and data loss is not presently a concern.

CC7.5 *The entity identifies, develops, and implements activities to recover from security incidents.*

The full infrastructure is defined in IaC, with Ansible automating deployment of supporting components; the environment can be torn down, hardened, and redeployed in roughly half an hour. Database backups will be necessary once the data store holds integral information, but no such data is currently stored.

CC8 — Change Management

CC8.1 *The entity authorizes, designs, tests, approves, and implements changes to infrastructure, data, software, and procedures.*

All code is tracked in version control and is readily restorable, and all changes are deployed through a build pipeline. These are acceptable change-management procedures for the current stage. Unit and integration testing is currently limited; expanding test coverage is assessed as medium priority, given the development-stage emphasis on reaching deployment and the fact that team-oriented readability and documentation are not yet a consideration for a solo project.

Additional Criteria

A1.2 (Availability) *Environmental protections, backup processes, and recovery infrastructure.*

Recovery is presently addressed through full IaC-based redeployment, as described under CC7.5: the environment can be reconstituted quickly from version-controlled definitions. Formal data backup and recovery processes are not yet implemented, consistent with the development-stage decision to defer durable storage until the application holds integral data. This is the principal open item for the Availability criteria and would be prioritized ahead of any production use.

C1.1 (Confidentiality) *The entity identifies and maintains confidential information.*

API keys are stored as secrets within the pipeline rather than in code. All long-lived AWS access keys have been deleted in favor of 2FA-authenticated users for access to critical infrastructure. Beyond these credentials, the system holds no confidential information at this stage.

PI1.x (Processing Integrity) *Scoping note.*

The Processing Integrity criteria concern the completeness, validity, accuracy, and authorization of system processing. These fall outside the security-focused scope of this assessment and depend on organizational commitments and system-processing requirements that are not yet formally defined for this project. They are therefore noted as out of scope rather than assessed.

Remediation Roadmap

The following summarizes each finding, its severity, and its current disposition. Statuses follow a three-state lifecycle — remediated (implemented and verified), implemented pending validation (change made, verification outstanding), and accepted or open (documented risk acceptance, or planned remediation not yet started). This snapshot-plus-roadmap presentation reflects a risk-based sequencing appropriate to a development environment; it is intentionally not a claim of full remediation.

Finding	Criterion / Area	Severity	Status & Next Step
Cluster SG open 0.0.0.0/0 to all ports	CC6.6	High	Implemented (pending validation) — Installation of AWS LB Controller via Helm to scope ingress rules written into Ansible; verify resolved SG rules + NodePort no longer directly reachable
Public subnets auto-assign public IP	CC6.6	High	Remediated — MapPublicIpOnLaunch set false in CloudFormation
NACLs permit admin ports (SSH/cluster)	CC6.6	High	Implemented (pending validation) — explicit deny entries added to IaC; confirm app + NAT return traffic unaffected
No API-change alerting on build S3	CC7.2	Med	Implemented (pending validation) — CloudWatch alerting added on build S3 bucket in the prebuild stage of the CloudFormation pipeline.
EKS node role over-permissioned	CC6.3	High	Open — remove debug-cycle inline policy; low effort, do next
CloudTrail not enabled	CC7.2	High	Open — enable one multi-region management-events trail to S3 (near-zero cost)
GuardDuty not enabled	CC7.2 / CC6.8	High	Remediated — recommend enabling foundational tier (single-digit $/mo at this scale)
VPC Flow Logs absent	CC7.2	Med	Open — build into CloudFormation w/ short retention or Glacier to bound

Finding	Criterion / Area	Severity	Status & Next Step
			cost
AWS Config recorder disabled	CC3.4 / CC7.1	Med	Accepted — per-config-item cost vs. EKS/CI-CD churn; compensating: Prowler + CFN drift detection
Security Hub not enabled	CC7.1	Low	Accepted — Config dependency drives same cost issue; Prowler covers point-in-time CSPM
IAM policies inline vs. grouped	CC6.3	Low	Open — move to Doggy App group for auditability as projects scale

---

Prepared as a self-assessment for portfolio purposes. Environment identifiers (account IDs, ARNs, resource names, IP addresses) should be sanitized before publication.